GIGLEND V3

User guide and manual test plan

How to run the website, how it works, and eleven hands-on tests covering lenders, normal borrowers, late payers and defaulters.

Part	What you get
Part 1 · Tutorial	Start the system locally, wallets and demo accounts, a guided tour of every page with screenshots, and how a loan moves from on-time to default.
Part 2 · Manual tests	Eleven short test cases. Each one says what it proves, who to log in as, the steps, what you should see, and a pass/fail box.

Prototype notice. GigLend runs on a local Hardhat test chain. KYC, OTP codes, income and gig payouts are mocks, and all keys in this guide are public Hardhat test keys. Never use them on a real network.

Prepared 25 October 2026 · branch `feature/auth-defaulter-handling`

PART 1

Tutorial: running and using GigLend

1. What GigLend is

GigLend is a community lending pool for gig workers. **Lenders** deposit ETH and receive GLP pool shares. **Borrowers** are verified members who take small installment loans that are repaid on the same weekly or two-weekly cycle as their gig payouts. The website covers the whole policy for handling defaults:

- **Prevent:** every member starts with an escrowed *credit-builder* loan; limits grow slowly; repayments must stay within 35% of income.
- **Detect early:** the backend watches payouts and warns before a payment is missed.
- **Cure cheaply:** a grace period without fees, one priced restructure per loan, autopay from a SafeRepay vault and an income sweep.
- **Recover from what is locked:** the borrower's stake, then a guarantor's stake, then the reserve, then lenders.
- **Book losses in steps:** late loans count for less in the pool's value as they get later.
- **Treat by type:** each default is classified as Hardship, Strategic or Fraud, with a workout, a lock-out, or a ban. The borrower can appeal within 21 days.
- **One person, one record:** a verified identity (stored only as a hash) is tied to one member record, so a new wallet cannot escape an old default.

2. The pieces

Part	Runs on	What it does
Local blockchain	Hardhat node · port 8545	Holds all contracts and money. Chain time can be moved forward for testing.
Smart contracts	Deployed by `npm run deploy:local`	Pool, loans, prevention rules, defaults, identity registry, credit score, reserve, SafeRepay vault, mock payout router.
Backend	Node/Express · port 4000	Sign-up wizard, OTP, wallet login, mock KYC, income check, attestation signing, notifier and inbox, admin and demo endpoints.
Website	React/Vite · port 5173	Everything the user sees. Calls the backend through `/api` and the contracts through the wallet.

3. Running it on your computer

Prerequisites: Node.js 20 or newer, npm, Git, and optionally the MetaMask browser extension. Run all commands from the project folder `blockchain project`.

One-time install

```
npm install
cd backend  && npm install && copy .env.example .env && cd ..
cd frontend && npm install && cd ..
```

Every time you start (four terminals, in this order)

#	Terminal	Command	Wait for
1	Blockchain	`npx hardhat node`	"Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545"
2	Deploy	`npm run deploy:local`	"Deployment complete." (this also writes the addresses into the website and backend)

3	Backend	`cd backend`
		`npm run db:reset`
		`npm run dev`
			"listening on http://localhost:4000". Keep this terminal visible: OTP codes are printed here.

4	Website	`cd frontend` `npm run dev`	The local URL, then open **http://localhost:5173**

> **Why db:reset?** The Hardhat node forgets everything when it stops. Resetting the backend database after a fresh chain removes old accounts that point at wallets the new chain does not know.

Load the demo data (optional but recommended)

With all four terminals running, open a fifth terminal:

```
cd backend && npm run seed:demo && cd ..
npm run demo:scenarios
```

`seed:demo` signs up five accounts through the real sign-up flow. `demo:scenarios` then lends 20 ETH and moves chain time forward about 130 days. It leaves one Hardship, one Strategic and one Fraud defaulter and a borrower in good standing.

4. Wallets and demo accounts

Every action that moves money is signed by a wallet. You can use either option below.

- **Dev wallet (easiest, no extension):** add `?devwallet=N` to any address, e.g. `http://localhost:5173/login?devwallet=3`. The page then uses Hardhat test account N. It works only with the local development server. Use `?devwallet=off` to switch it off.
- **MetaMask:** add a network with RPC `http://127.0.0.1:8545`, chain ID `31337`, currency ETH. Import the private key of the account you want from the table below. After restarting the node, use MetaMask → Settings → Advanced → *Clear activity tab data*.

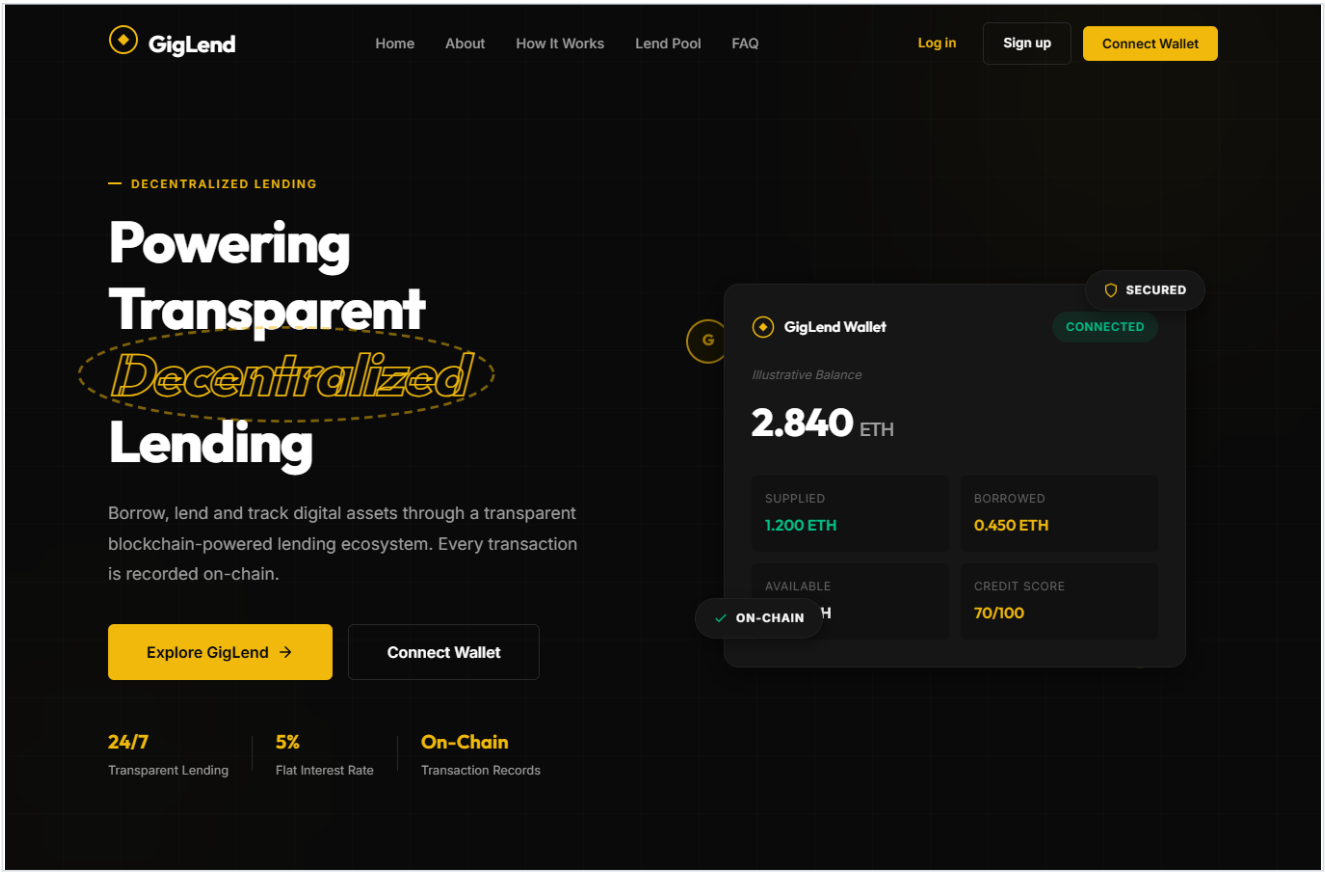
Account	Demo login (password: demo-password-1)	Role after demo:scenarios	Private key (Hardhat test key)
#0	f39fd6@demo.giglend.test	Ops Admin (admin + dispute roles), lender	`0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80`
#3	90f79b@demo.giglend.test	Asha · Hardship defaulter on a workout plan	`0x7c852118294e51e653712a81e05800f419141751be58f605c371e15141b007a6`
#4	15d34a@demo.giglend.test	Vikram · Strategic defaulter	`0x47e179ec197488593b187f80a00eb0da91f1b9d0b13f8733639f19c30a34926a`
#5	996550@demo.giglend.test	Ravi · Fraud, permanently barred	`0x8b3a350cf5c34c9194ca85829a2df0ec3153be0318b5e2d3348e872092edffba`
#7	14dc79@demo.giglend.test	Meena · good borrower with an open loan	`0x4bbbf85ce3377467afe5d46f804f221813b2bb87f24d81f60f1fcdbf7cbf4356`

Account	Demo login (password: demo-password-1)	Role after demo:scenarios	Private key (Hardhat test key)
#10–#19	(sign up yourself)	Fresh wallets for new test users	Printed by `npx hardhat node`

5. A tour of the website

Home page

The landing page explains GigLend. The top bar has **Log in**, **Sign up** and **Connect Wallet**. Lending is open to anyone with a wallet; borrowing needs a verified account.

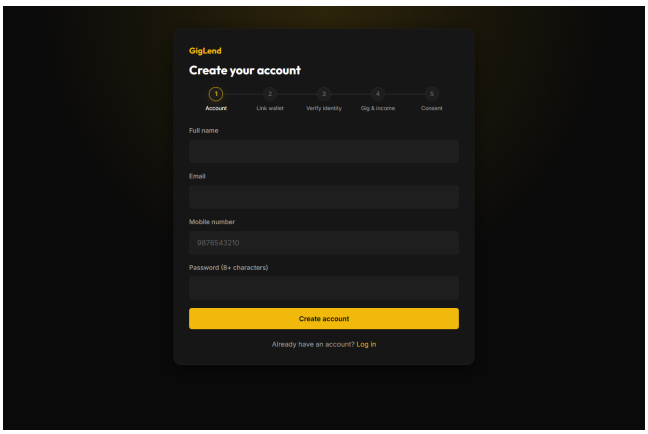


Landing page with the new Log in / Sign up buttons.

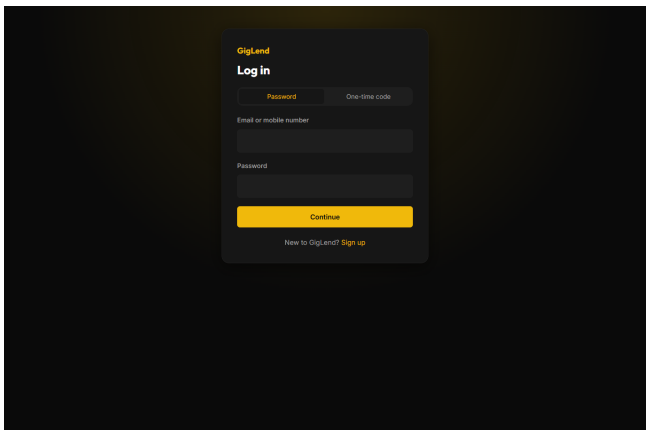
Signing up (five steps)

The sign-up wizard saves each step, so you can log in later and continue where you stopped.

1. **Account:** name, email, mobile and password. A 6-digit code is printed in the *backend terminal* (look for `[mock-otp] ... code=123456`). Enter it to verify.
2. **Link wallet:** sign a message with your wallet (Sign-In with Ethereum). It costs nothing and sends no transaction.
3. **Verify identity:** a PAN-format string (5 letters, 4 digits, 1 letter, e.g. `ABCDE1234F`), the *last 4* digits of Aadhaar, and any selfie image. Only a salted hash of the PAN ever reaches the blockchain.
4. **Gig and income:** platform, worker ID, how you are paid (weekly or every two weeks), 3–6 months of payouts in rupees, and every existing EMI.
5. **Consent:** data use (required), autopay (on by default) and lending-circle visibility (optional). Then press **Register on-chain**.



Step 1 of the wizard with the progress bar.



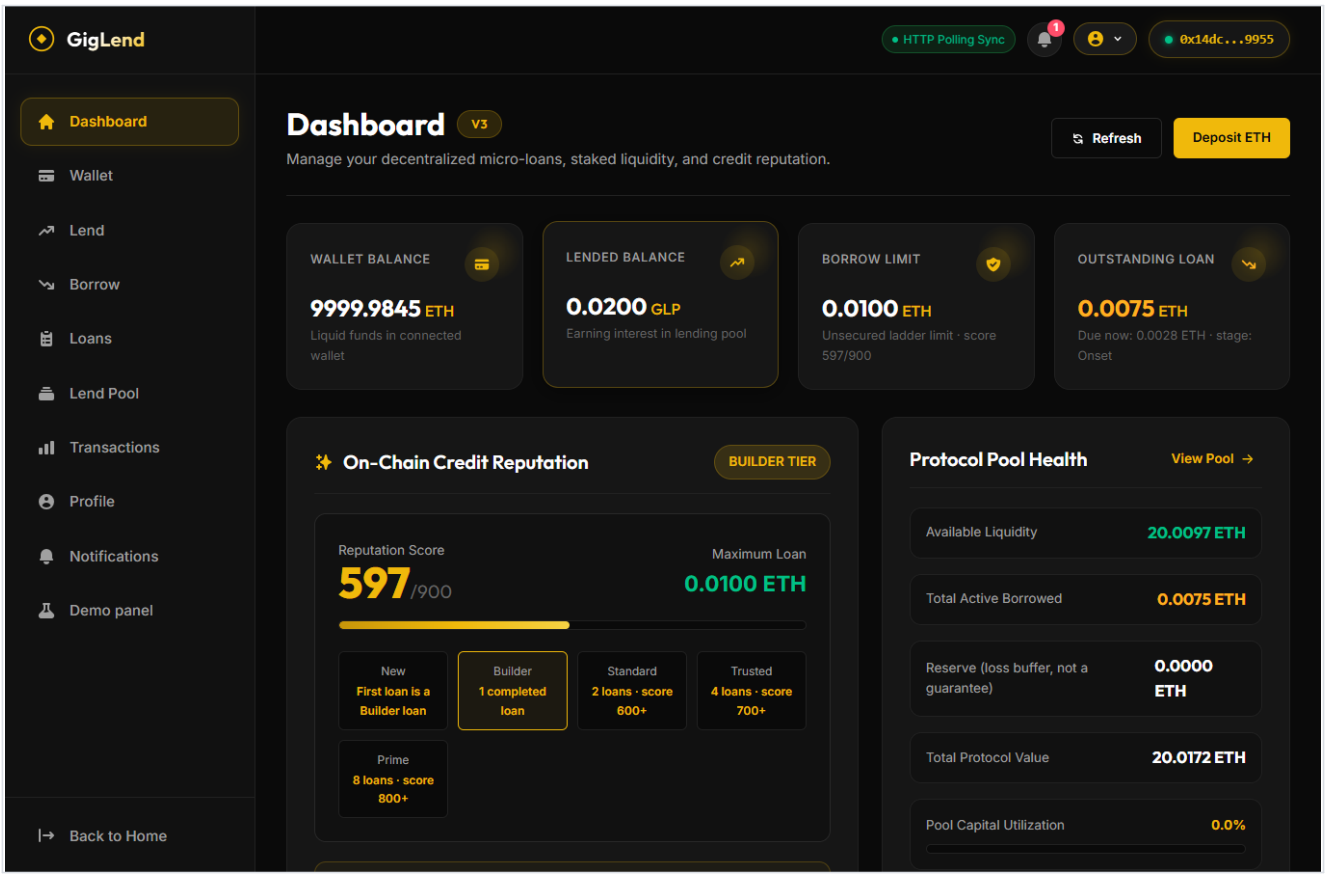
Log in: password or one-time code, then a wallet signature.

Logging in

Enter your email or mobile and password (or ask for a one-time code), then press **Sign in with Ethereum**. The signature must come from the wallet linked to your account. If the browser wallet later switches to a different account, a red banner appears and every button is disabled until you switch back.

Dashboard

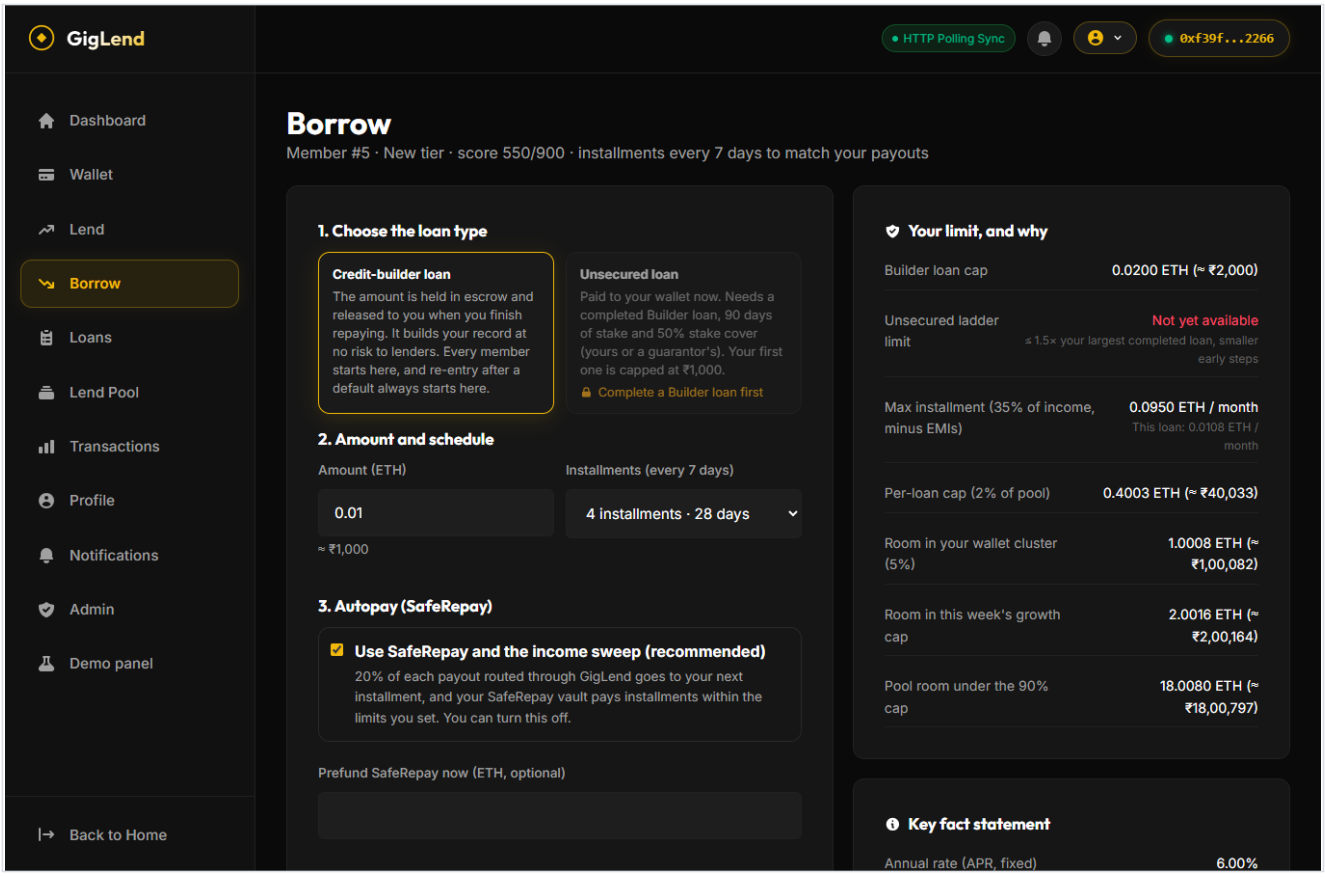
Shows wallet balance, GLP (pool shares), your current borrowing limit and why, any open loan with its stage, your credit score on the 300–900 scale and your tier (New → Builder → Standard → Trusted → Prime). The bell at the top right shows unread messages.



Meena's dashboard: score 597/900, Builder tier, an open loan in the Onset stage, one unread notice.

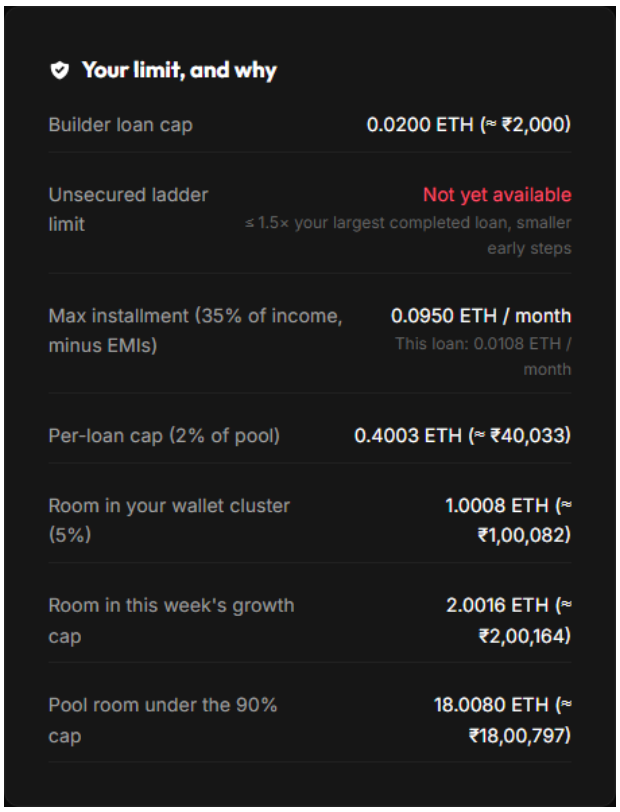
Borrow

Choose the loan type, amount and number of installments. The right side explains your limit line by line and shows a **key fact statement** (rate, installment, total cost, late fee, grace period, penalty-free exit) that is priced live. If a rule blocks the loan, the reason appears there before you submit.

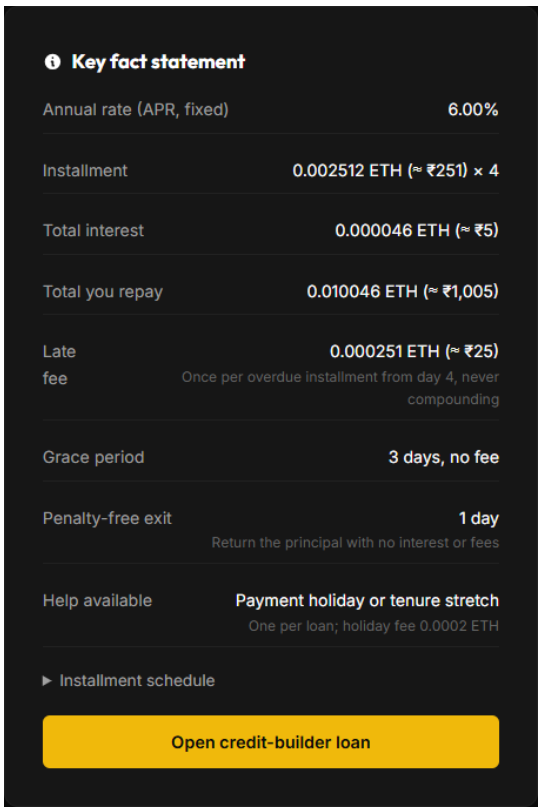


Borrow page for a new member: only the credit-builder loan is available until one is repaid.

- **Credit-builder loan:** the money is held in escrow and paid to you only after the last installment. It builds your record at no risk to lenders.
- **Unsecured loan:** paid to your wallet now. It needs one completed builder loan, 90 days of pool stake, and stake covering 50% of the loan (yours, or pledged by a guarantor). The first one is capped at ₹1,000 (0.01 ETH).
- While any loan is open, your GLP stake is locked: you cannot withdraw or transfer it.



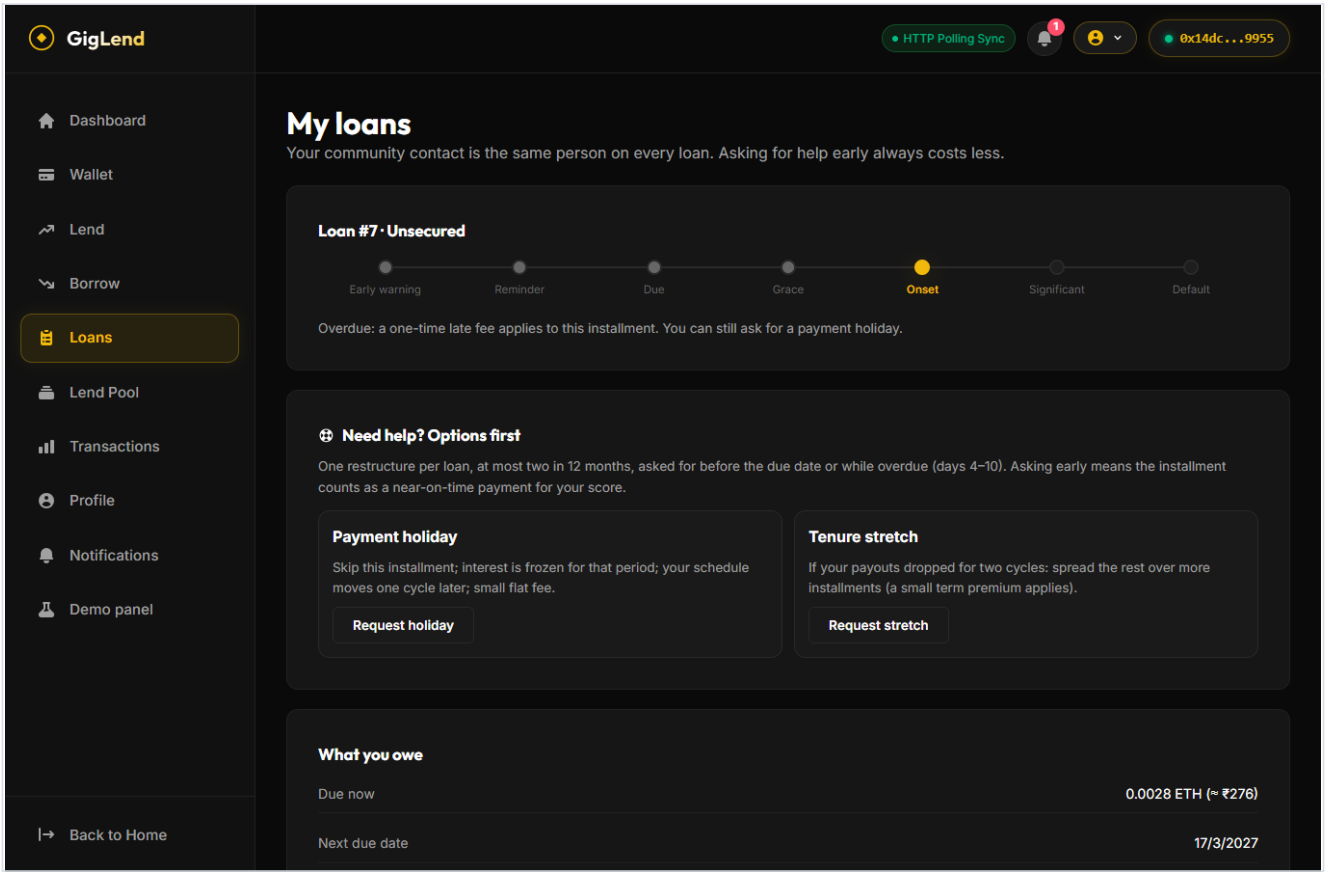
"Your limit, and why": each cap is listed separately.



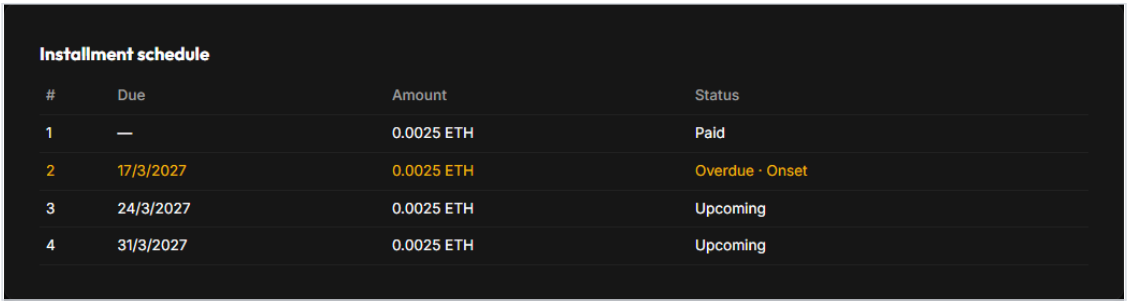
Key fact statement, priced live from the contracts.

Loans

The stage timeline shows where your loan is. Help options (payment holiday, tenure stretch) are shown *before* the amount you owe. Below them: what you owe, pay buttons, the installment schedule, your SafeRepay vault, and loan history.



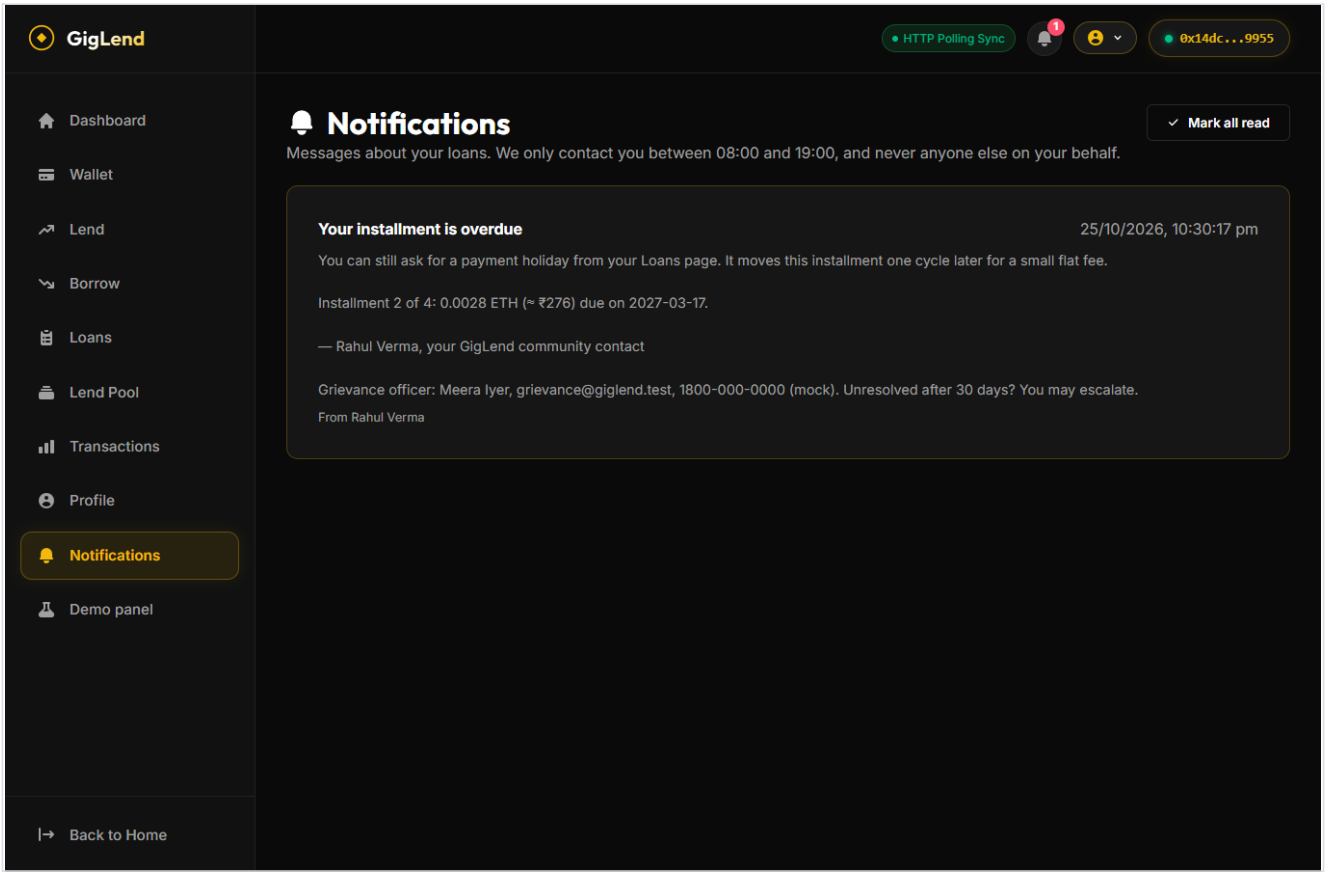
An overdue loan in the Onset stage, with restructure options shown first.



Installment schedule: paid, current (overdue · Onset) and upcoming installments.

Notifications

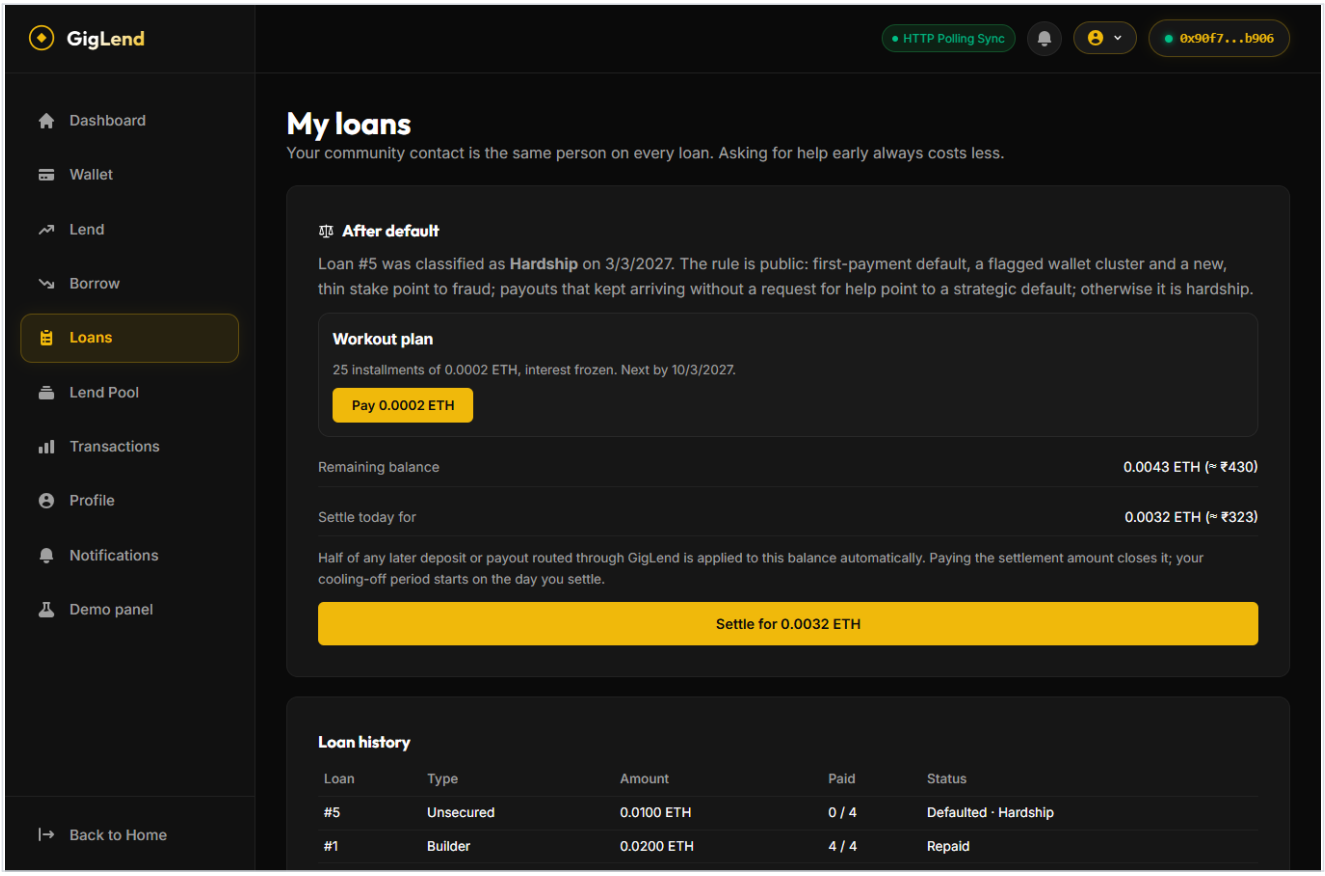
Messages come from one named community contact, who stays the same on every loan. Each message offers help first, then states the amount and date, and names the grievance officer. Messages are only sent between 08:00 and 19:00 in the member's time zone; anything created outside those hours waits in a queue.



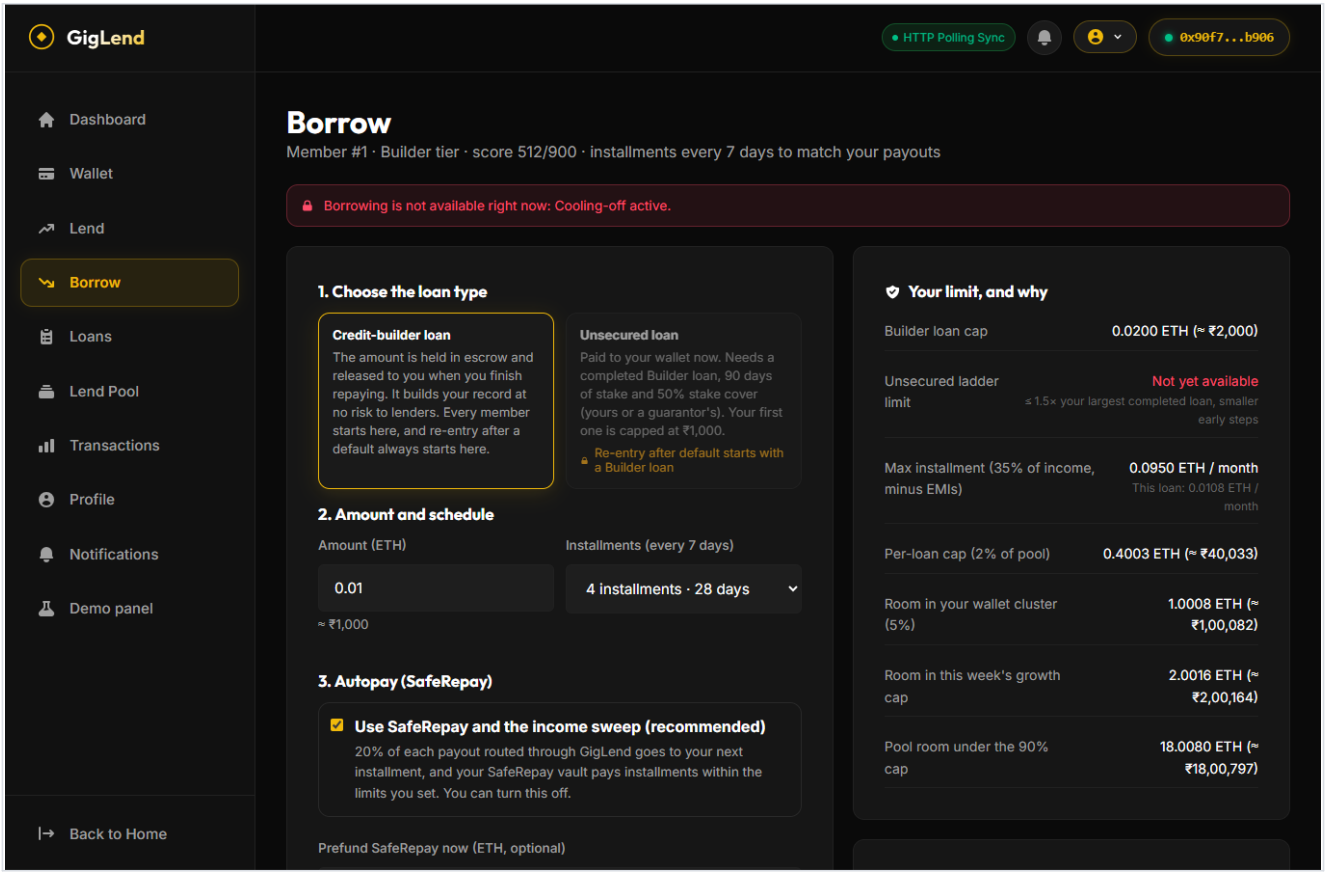
An overdue notice from the member's contact, with the grievance line.

After a default

When a loan is marked as defaulted, the Loans page shows the classification and the reason rule, the appeal deadline, the remaining balance, a discounted **settle today** amount, and (for Hardship) a workout plan with small installments and no interest. Borrowing stays blocked until the balance is settled; the cooling-off period starts on the day you settle.



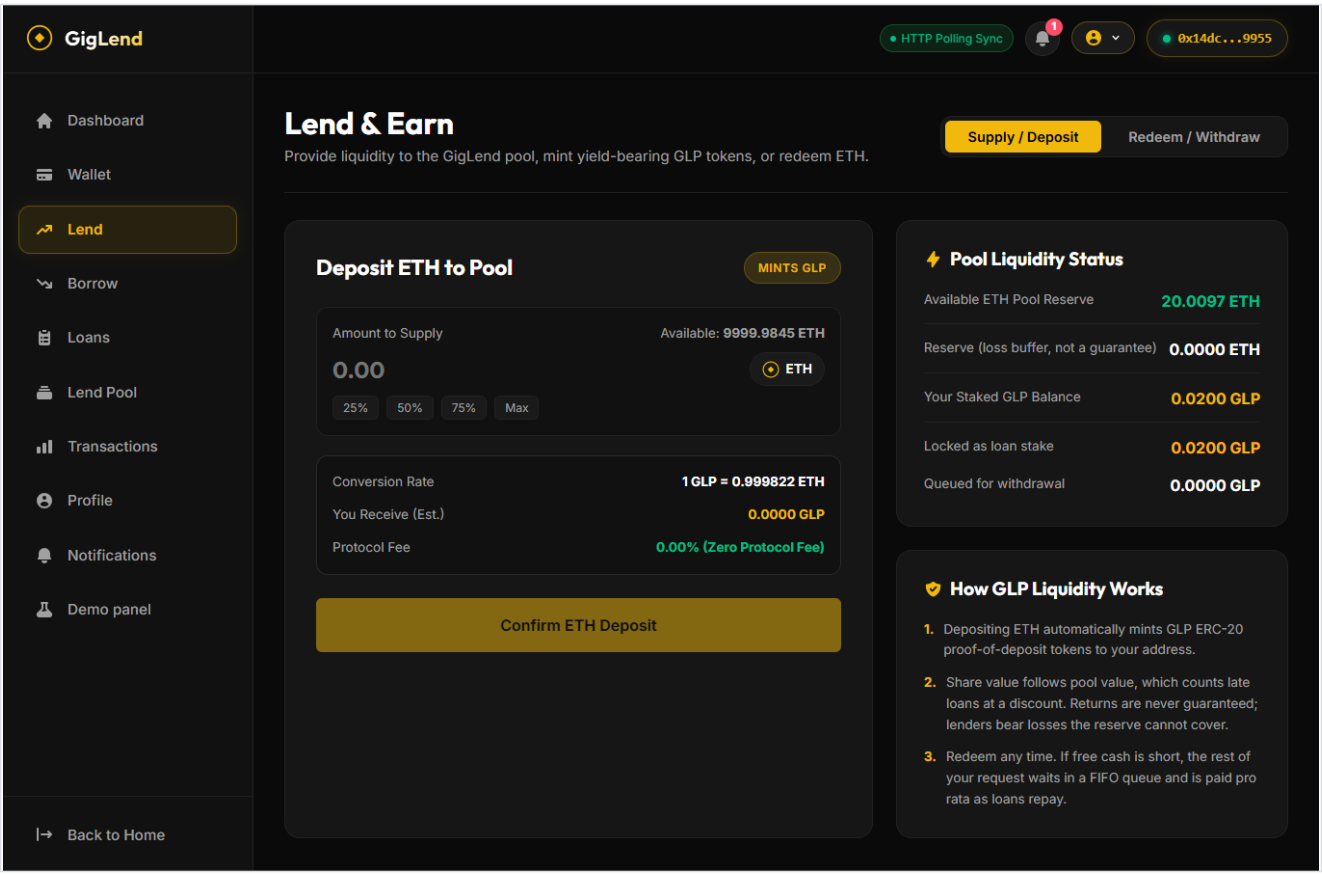
Asha (Hardship): workout plan, remaining balance and the settlement quote.



Borrow page for a defaulter: "Cooling-off active", and re-entry must start with a builder loan.

Lend

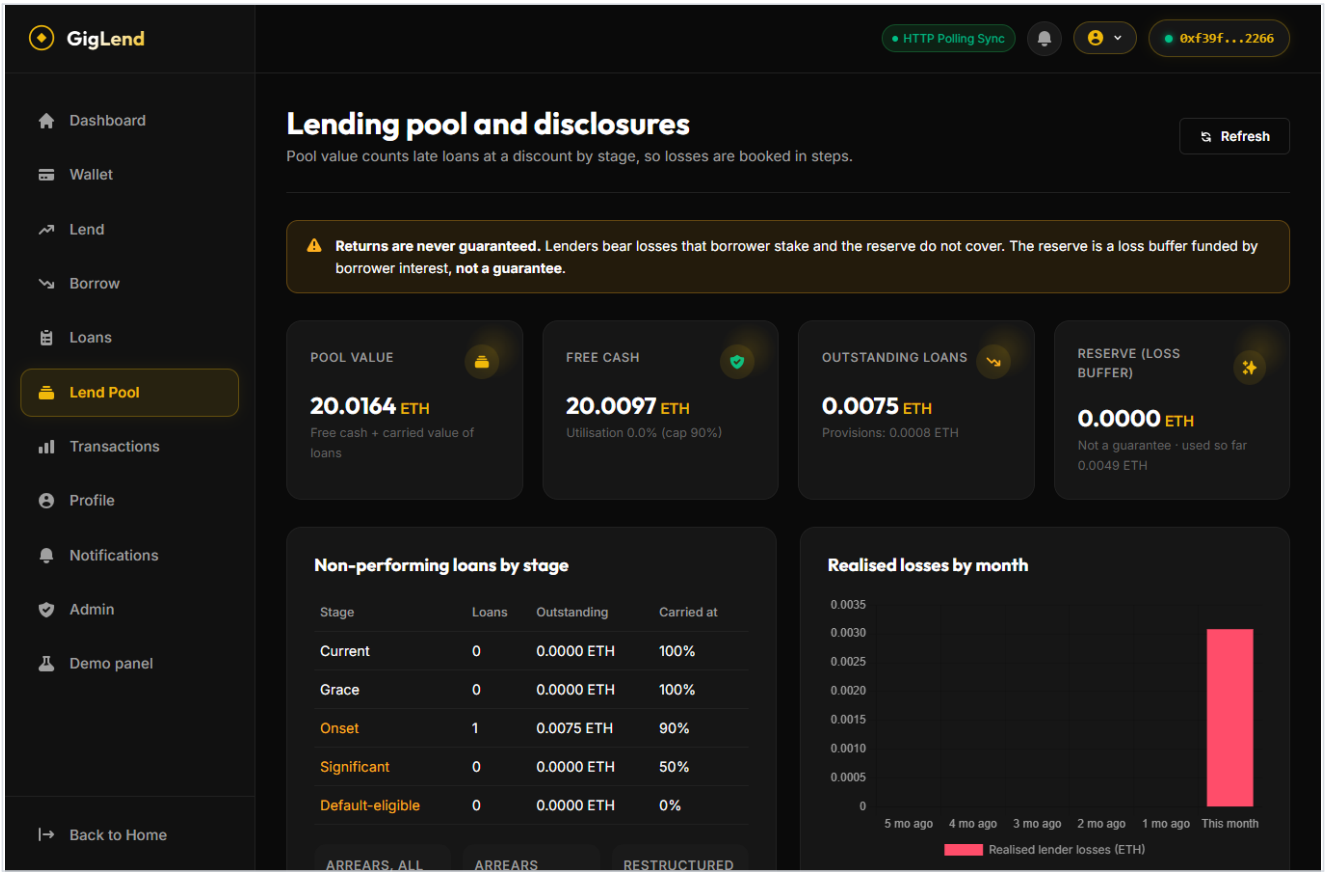
Deposit ETH to receive GLP shares and withdraw them later. The page shows the live share price, how much of your GLP is locked as loan stake, and any withdrawal still waiting in the queue. If the pool's free cash is short, a withdrawal joins a first-in-first-out queue and is paid as loans repay.



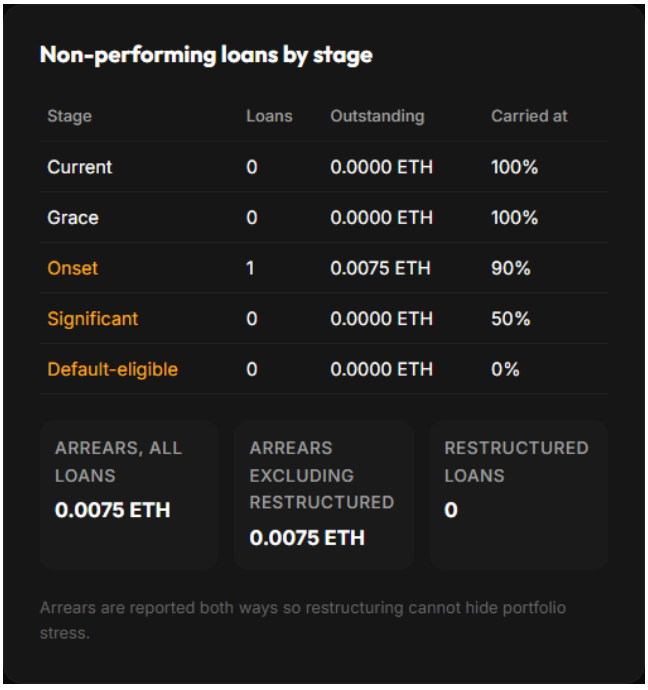
Lend page: share price, reserve (a loss buffer, not a guarantee) and locked stake.

Lend Pool (disclosures)

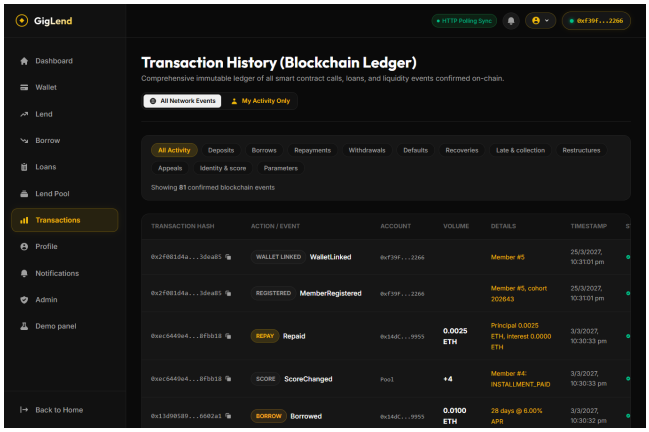
The pool page shows the pool value (late loans count for less), free cash, outstanding loans, the reserve, loans grouped by stage, arrears reported twice (with and without restructured loans), losses by month, and your queue position. It states plainly that returns are never guaranteed.



Pool disclosures.



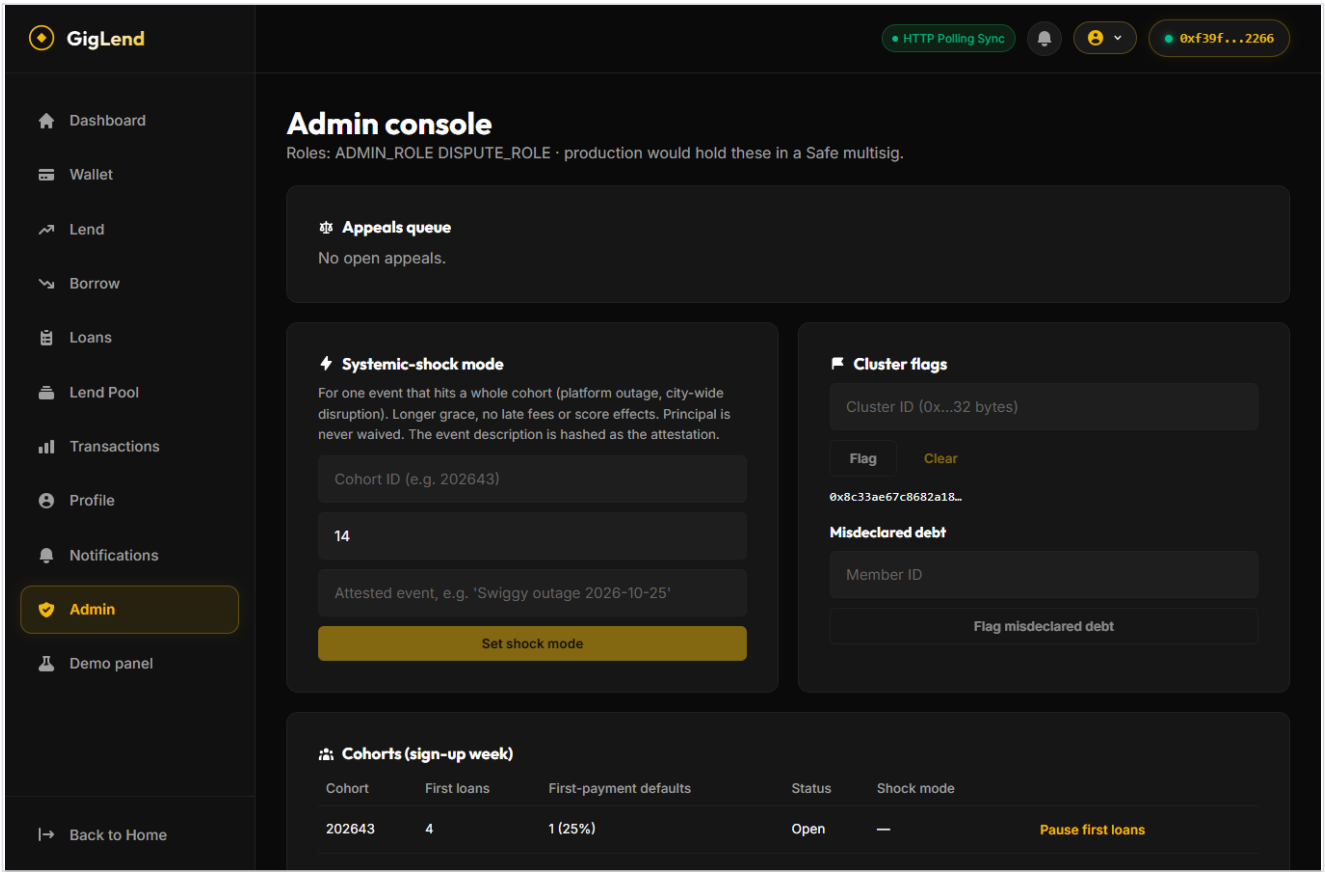
Loans by stage and how much each counts for.



Transactions ledger: every contract event with filters.

Admin console

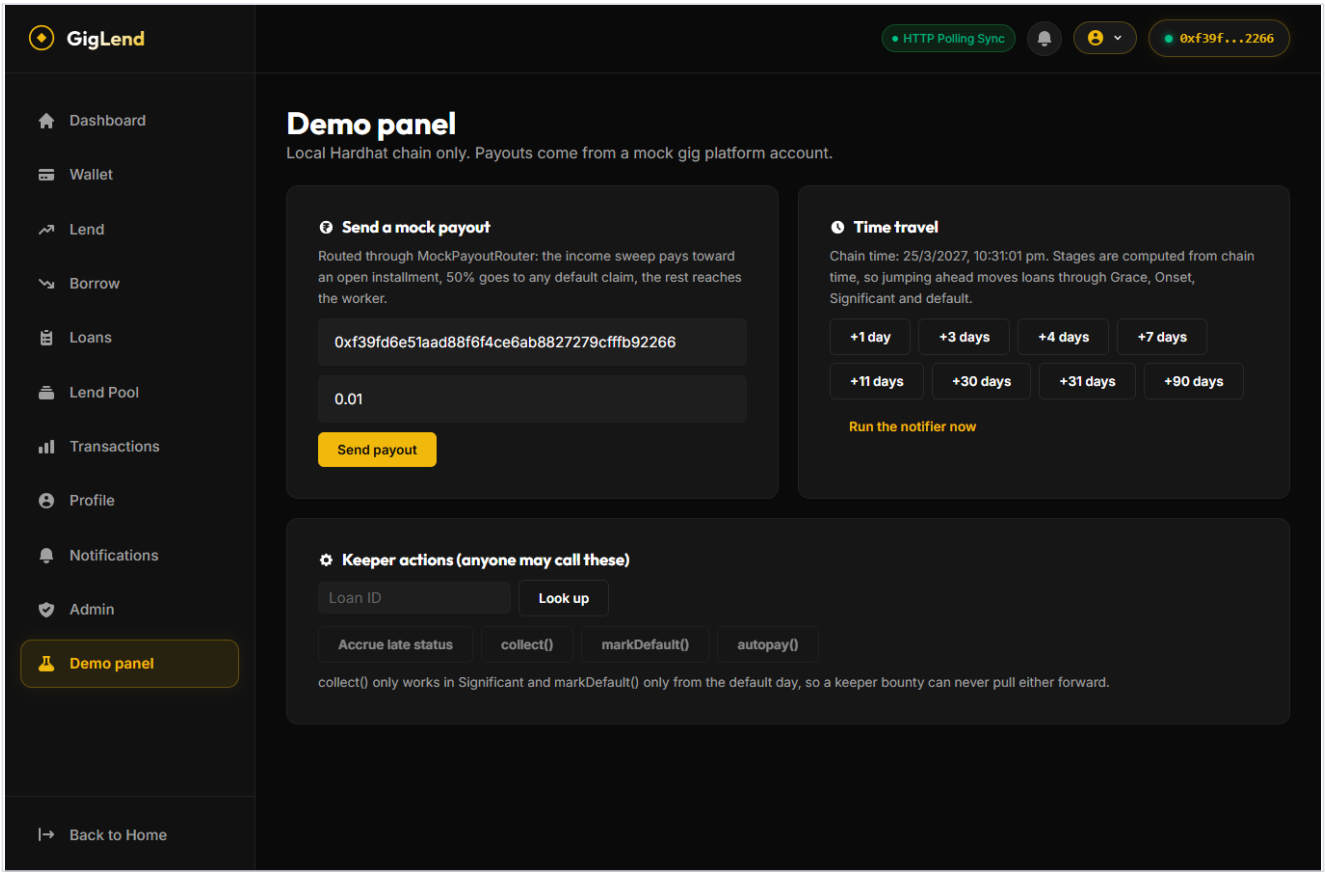
Only visible to the Ops Admin account. It holds the appeals queue (reclassify a default), systemic-shock mode for a cohort, wallet-cluster flags, the misdeclared-debt flag, cohort pause status, every policy parameter with its allowed range, and the message log.



Admin console: roles, appeals, shock mode and cluster flags.

Demo panel

Local-only tools for testing: send a mock gig payout to any wallet, move chain time forward by 1–90 days, run the notifier now, and run keeper actions (`accrueLate`, `collect`, `markDefault`, `autopay`) on any loan number.



Demo panel: mock payouts, time travel and keeper actions.

6. How a loan's life works

Stages are calculated from the due date of the unpaid installment. Nothing has to run in the background to move a loan between stages.

Stage	When (days after the due date)	What happens	Counts in pool value at
Early warning / Reminder	7 and 3 days *before* due	Inbox nudge if payouts are falling or SafeRepay is too low to cover the installment.	100%
Grace	Days 1–3 (7 for Trusted/Prime members or loans over 3 months)	Autopay retries. No fee, no score penalty, no personal contact.	100%
Onset	Days 4–10	One flat late fee for this installment (never compounds). Score penalty. Notice sent. Payment holiday still allowed.	90%
Significant	Days 11–29 (to 44 for long loans)	Anyone can call collect(): locked stake pays the installment and the caller earns a small bounty. Guarantor alerted.	50%
Default	Day 30 (45 for long loans)	Anyone can call markDefault(): classification, then the loss waterfall, then a residual claim.	0%

Loss waterfall at default: **1.** borrower's locked stake → **2.** guarantor's stake → **3.** reserve fund → **4.** lenders, through a lower share price. Whatever is still unpaid stays as a claim on the member. Half of any later deposit or payout goes to that claim automatically.

Type	Typical signals	Settlement discount	Cooling-off after settlement
Hardship	Seasoned member, income fell or asked for help	25%, plus an interest-free workout plan	30 days
Strategic	Payouts kept arriving after the miss, never asked for help	5%	12 months
Fraud	Two of: first-payment default, a new or thin stake, a flagged wallet cluster	None	Never (permanently barred)

7. Time-travel cheat sheet

Use **Demo panel** → **+N days** (or `node scripts/timeTravel.js advance N`). Time travel moves the whole chain, so every demo loan ages together. For a **new weekly loan with nothing paid**, the first installment is due 7 days after it opens:

Move forward from the moment the loan opened	Stage of the first installment
+6 days	Active (reminder window)
+8 days	Grace (no fee)
+11 days	Onset (late fee charged once)
+18 days	Significant (collect allowed)
+37 days	Default-eligible (markDefault allowed)

> Other handy commands: `node scripts/timeTravel.js status <loanId>` prints a loan's stage. `node scripts/timeTravel.js 31` moves 31 days and defaults every eligible loan.

PART 2

Manual tests

Eleven short tests that cover the main kinds of user: lenders, new members, normal borrowers, late payers, borrowers who ask for help, and each type of defaulter. Run them in order on a fresh start (section 3, including `seed:demo` and `demo:scenarios`). Tests that sign up new users use dev-wallet accounts #10–#19.

> **Before you begin:** keep the backend terminal visible (OTP codes). Time travel affects every loan on the chain, so finish one test's time jumps before starting the next. If a page shows stale numbers after a jump, press Refresh or reload.

Test summary

#	Test	User type
T1	Lender: deposit, share price, withdrawal and locked stake	Lender
T2	New member sign-up and on-chain registration	New member
T3	Login security: password + wallet, wrong wallet, wallet switch	Any member
T4	Normal borrower: credit-builder loan from start to finish	Normal borrower
T5	Normal borrower: first unsecured loan and the limits	Normal borrower
T6	Late payer: grace, one late fee, then collect from stake	Late payer
T7	Borrower who asks for help: payment holiday and its limits	Borrower asking for help
T8	Defaulters (Hardship): default timing, stake recovery, workout, settlement	Defaulter (Hardship)
T9	Defaulters (Strategic and Fraud) and the appeal	Defaulters (Strategic, Fraud) + admin
T10	Same person, new wallet: the default follows the identity	Defaulter with a new wallet
T11	Admin and safety: shock mode, parameters, notices and disclosures	Admin / everyone

T1	Lender: deposit, share price, withdrawal and locked stake
Proves	Lending is open to everyone, shares are priced from pool value, and stake behind an open loan cannot leave.
Log in as	Any wallet for deposits (e.g. `?devwallet=12`); Meena (#7) for the locked-stake check.
How to test	1. Open **Lend**, deposit **1 ETH** and confirm. 2. Check the GLP received is about 1 ÷ share price, and that **Lend Pool** shows pool value up by about 1 ETH. 3. Switch to **Redeem / Withdraw**, withdraw 0.5 GLP. 4. Log in as Meena (open loan) and try to withdraw all her GLP.
Expected	Deposit and withdrawal succeed and appear in Transactions as Deposited / Withdrawn. Meena's withdrawal is refused: "Part of your GLP is locked as stake for an open loan or guarantee." Her Lend page shows the locked amount.
Result	[ ] Pass [ ] Fail Notes: _____

T2	New member sign-up and on-chain registration
Proves	The five-step wizard works, bad input is refused, and the member is registered on-chain.
Log in as	A new user with `http://localhost:5173/signup?devwallet=10`.
How to test	1. Create the account. Enter a *wrong* code first, then the real code from the backend terminal. 2. Press **Connect**, then **Sign and link wallet**. 3. In **Verify identity**, try a 12-digit Aadhaar number first, then use only 4 digits, a new PAN (e.g. `TESTA1234Z`) and any image. 4. Fill in income (3 months, weekly) with one EMI. On the consent step, try to continue without data-use consent, then tick it. 5. Press **Register on-chain**, then open **Profile**.
Expected	Wrong code is rejected ("That code is not correct"). A full Aadhaar is rejected. Consent cannot be skipped. Profile shows: contact verified, linked wallet, masked PAN, on-chain member number, tier *New*, score 550, a named community contact, and the income band. Transactions shows MemberRegistered.
Result	[ ] Pass [ ] Fail Notes: _____

T3	Login security: password + wallet, wrong wallet, wallet switch
Proves	A session is valid only when the signing wallet is the linked wallet.
Log in as	The member from T2.
How to test	1. Log out (profile menu → Log out). Open `/login?devwallet=11`, enter the T2 email and password, press **Sign in with Ethereum**. 2. Open `/login?devwallet=10` and log in again with the correct wallet. 3. Try the **One-time code** tab: request a code, enter it from the backend terminal. 4. (MetaMask only) After logging in, switch MetaMask to another account.
Expected	With wallet #11 the login is refused (wrong wallet). With #10 you reach the Dashboard. The one-time code works the same as the password. With MetaMask switched, a red banner appears and buttons are disabled.
Result	[ ] Pass [ ] Fail Notes: _____

T4	Normal borrower: credit-builder loan from start to finish
Proves	Credit-builder first; money stays in escrow until fully repaid; good repayment raises the score.
Log in as	The member from T2 (logged in with devwallet 10).
How to test	1. Open **Lend** and deposit **0.02 ETH** as stake (needed later for T5). 2. Open **Borrow**. Check the Unsecured option shows "Complete a Builder loan first". Read the key fact statement. 3. Open a **credit-builder** loan of 0.01 ETH with 2 installments (untick autopay to keep it simple). 4. On **Loans**, check "Held in escrow for you" = 0.01 ETH and your wallet did not receive it. Press **Pay** for each installment.
Expected	After the last payment: "Loan fully repaid", the 0.01 ETH escrow arrives in your wallet, Loan history shows *Repaid 2/2*, the score rises (550 → 573) and the tier becomes *Builder*. While the loan was open, Borrow showed "You have an open loan" instead of a new application.
Result	[ ] Pass [ ] Fail Notes: _____

T5	Normal borrower: first unsecured loan and the limits
Proves	Unsecured loans need 90 days of stake, 50% cover and the ₹1,000 first cap; limits explain themselves.
Log in as	The same member after T4.
How to test	1. On **Borrow**, note the hold message "Stake must be held for 90 days". 2. **Demo panel → +90 days.** (Your stake has to be deposited *before* this jump; T4 step 1 did that.) 3. Back on **Borrow**, choose Unsecured and enter 0.02 ETH. Read the red message in the key fact statement. Change it to 0.01 ETH, keep autopay on and open the loan. 4. Open **Lend** and look at locked GLP.
Expected	0.02 ETH is refused with "Above ladder limit". 0.01 ETH (₹1,000) is paid straight to your wallet. All your GLP shows as locked. The limit panel shows the 35% income cap and the other caps with their remaining room.
Result	[ ] Pass [ ] Fail Notes: _____

T6	Late payer: grace, one late fee, then collect from stake
Proves	No fee during grace; one non-compounding fee in Onset; stake is applied only from Significant, with a keeper bounty.
Log in as	The member from T5 with the open unsecured loan (note its loan number on Loans).
How to test	1. Demo panel **+8 days**: Loans shows *Grace*; "Due now" equals the installment (no fee). 2. **+3 days** (Onset): "Due now" grows by one small late fee. Demo panel → **Accrue late status**. Transactions shows InstallmentMissed and LateFeeCharged once. 3. **+4 days**: the fee is unchanged. Try Demo panel → **collect()**: it fails. 4. **+4 days** (Significant): press **collect()** again.
Expected	Grace has no fee and no score penalty. The fee is charged once and never grows. collect() fails with "Not in Significant stage" before day 11, then succeeds: Transactions shows StakeApplied and BountyPaid, and installment 1 is marked paid from the locked stake.
Result	[ ] Pass [ ] Fail Notes: _____

T7	Borrower who asks for help: payment holiday and its limits
Proves	One priced restructure per loan, allowed before due or in Onset, never twice.
Log in as	The member from T5/T6. After T6 their second installment is a few days overdue (Onset).
How to test	1. On **Loans**, note the stage (*Onset*) and the next due date. Press **Request holiday**. 2. Check the next due date moved one cycle later, the stage is back to on-time, and a small holiday fee was added to "Due now". 3. Press **Request holiday** again. 4. Open **Lend Pool** and read the restructured-loans and arrears figures.
Expected	The first request succeeds (RestructureGranted in Transactions). The second fails with "Loan already restructured". The pool page shows *Restructured loans: 1*, and arrears are reported both with and without restructured loans.
Result	[ ] Pass [ ] Fail Notes: _____

T8	Defaulters (Hardship): default timing, stake recovery, workout, settlement
Proves	Default only from day 30; locked stake is used first; hardship gets a workout and a 25% discount; cooling-off starts at settlement; re-entry starts with a builder loan.
Log in as	The member from T7, Meena (#7, loan #7) and Asha (#3).
How to test	1. Demo panel → **markDefault()** on the T7 member's loan (now on time after the holiday). 2. Meena's loan #7 is now long overdue after the earlier time jumps. Run **markDefault()** on loan 7, then log in as Meena and open **Loans**. 3. Log in as Asha. On **Loans**, read the classification, remaining balance and "Settle today" amount (about 75%). Pay one workout installment, then press **Settle**. 4. Open **Borrow** as Asha. Then Demo panel **+31 days** and open Borrow again.
Expected	Step 1 fails with "Default day not reached". Meena's default succeeds as *Hardship*: her locked stake covered the whole debt (lender loss 0 in Transactions), so there is no balance and Loans shows a 30-day cooling-off date. For Asha, Borrow shows "Cooling-off active" after settling. 31 days later it opens again, but Unsecured says "Re-entry after default starts with a Builder loan".
Result	[ ] Pass [ ] Fail Notes: _____

T9	Defaulters (Strategic and Fraud) and the appeal
Proves	Classification follows the public rule, fraud is barred and its wallet cluster flagged, and an appeal lets the admin reclassify.
Log in as	Vikram (#4) to observe Strategic; a new user with `?devwallet=14` for Fraud; Ops Admin (#0).
How to test	1. As Vikram, open **Loans**: the type is *Strategic* (payouts kept arriving after the miss) and "Settle today" is about 95% of the balance. 2. Sign up a new user with devwallet 14. Do **not** deposit anything. Open a credit-builder loan of 0.02 ETH and never pay. Demo panel **+37 days**, then **markDefault()** on that loan. 3. As the new user, open **Borrow** (blocked) and **Loans**, and press **Appeal**. 4. As Ops Admin, open **Admin**: see the flagged cluster, then press → **Hardship** on the appeal. Reload the new user's Borrow and Loans pages.
Expected	Vikram: Strategic, 5% discount, 12 months of cooling-off after settlement. New user: classified *Fraud* (first-payment default and no stake). Borrow says "Permanently barred" and Admin lists the cluster as flagged. After reclassification, Transactions shows Reclassified (Fraud → Hardship); the user is no longer barred (Borrow now says "Cooling-off active" until the small balance is settled) and the workout option appears.
Result	[ ] Pass [ ] Fail Notes: _____

T10	Same person, new wallet: the default follows the identity
Proves	One real person = one record; a fresh wallet cannot escape an old default.
Log in as	Vikram (#4, unsettled Strategic default) and a new sign-up with `?devwallet=13`.
How to test	1. Sign up a new account with devwallet 13. At the identity step use Vikram's PAN `VIKRM5678B`. 2. Log in as Vikram and press **Profile** → **Unlink this wallet** (allowed because he has no open loan). 3. Log back in as the new account (it resumes at the identity step), enter `VIKRM5678B` again, finish the wizard and register on-chain. 4. Open **Profile**, **Loans** and **Borrow** with the new wallet.
Expected	Step 1 is refused: "This identity already has an active wallet...". After unlinking, the new wallet joins Vikram's record ("This identity already has a GigLend record..."). Profile shows Vikram's member number. Loans shows his Strategic default and balance, and Borrow says "Cooling-off active".
Result	[ ] Pass [ ] Fail Notes: _____

T11	Admin and safety: shock mode, parameters, notices and disclosures
Proves	Shock mode protects a whole cohort without waiving principal; parameters are bounded; contact rules and disclosures hold.
Log in as	Ops Admin (#0) plus the T5 member (the only open loan left).
How to test	1. In **Admin** → **Cohorts**, note the borrower's cohort. Set **shock mode** for it: 14 days, event "Platform outage test". 2. Using the T5 member's open loan (same cohort), press Demo panel +1 day until Loans shows the installment is about 6 days late. Check Loans and Pool. 3. In **Policy parameters**, set `burdenBps` to 6000 and save; then set it to 4000. 4. Open the bell / **Notifications** and **Admin** → **Message log**. Then read the **Lend Pool** page.
Expected	Under shock mode the loan is still in Grace at day 6 with no late fee, and outstanding principal is unchanged. 6000 is refused ("Value out of bounds"), 4000 is saved (ParameterUpdated). Every notice is signed by the named contact and carries the grievance line. Notices created outside 08:00–19:00 show as "queued until 8:00". The pool page shows loans by stage, both arrears figures, losses by month, and "not a guarantee". After T9's fraud default, the Cohorts table shows the cohort as *Paused (automatic)* for new first loans.
Result	[ ] Pass [ ] Fail Notes: _____

Sign-off

Tester	Date	Build / commit	Tests passed	Notes
			/ 11	

Troubleshooting

Symptom	Fix
"Connect Your Wallet" on every page	Add `?devwallet=N` to the address, or unlock MetaMask on network 31337.
Login says the wallet is wrong	The browser wallet is not the one linked to that account. Use the account number from the table in section 4.
"Registry unavailable" or zero balances	The node was restarted. Run `npm run deploy:local`, then `npm run db:reset` and restart the backend.
"Attestation expired" after big time jumps	Profile → Refresh attestation (it signs a new one valid for 365 chain days).
MetaMask "nonce too high"	MetaMask → Settings → Advanced → Clear activity tab data.
No notices in the inbox	Outside 08:00–19:00 they wait in the queue; check Admin → Message log, or press "Run the notifier now" on the Demo panel.
Too many requests (rate limited)	Wait one minute; sign-up and login allow 60 requests per minute.